

Project 2: Building Agentic Applications

Patchandi Yugan Kumar A0264684R, Low Bi Shen, Gary A0252025U,

Chua Yuan Cheng A0258734U, Chong Wei Jie A0252004Y

AI Workflow Group 6

{e1032496, e0957613, e0969706, e0957592}@u.nus.edu

Abstract	2
Features implemented	2
Introduction	3
End to End System Design and Architecture	5
Chatbot Service	5
Backend Service	7
Workflow Generation Service	7
Workflow Execution Engine	8
Dependency-aware scheduling and parallel execution	8
Prefect, Redis, and multi-tenancy distributed execution	10
Fault tolerance and error handling	11
Workflow Generator LLM Fine-tuning	11
Dataset construction	11
Supervised Fine-tuning	13
Evaluation	14
Fine-tuning	14
Workflow Execution Engine	17
Conclusion	18
Analysis and Evaluation of Other Groups during the Project Showcase	18
References	22
Appendix	23

Abstract

Recent advances in large language models (LLMs) have enabled a shift from passive question-answering systems to agentic workflows that execute complex, multi-step tasks from natural language instructions. However, existing workflow automation platforms such as n8n still rely heavily on manual workflow construction and lack efficient, adaptive execution mechanisms. This creates a gap between high-level user intent and practical, scalable workflow deployment. In this work, we present an end-to-end agentic workflow system that bridges this gap by integrating LLM-based workflow generation with an optimized execution engine. Our system enables users to specify automation tasks in natural language, iteratively refine workflows through multi-turn interaction, and directly edit workflow graphs via a visual interface. To support this, we construct a dataset of structured workflow representations and fine-tune a 3B-parameter model for prompt-to-workflow generation, incorporating both single-turn and multi-turn editing examples. On the systems side, we design a distributed execution engine that transforms generated workflows into dependency-aware task graphs, enabling parallel execution, fault tolerance, and multi-tenant scalability using a microservice architecture. Empirical evaluation shows that the fine-tuned model achieves perfect JSON validity and schema compliance, while highlighting challenges in modeling correct inter-node dependencies. We further analyze the limitations of reinforcement learning approaches for structured workflow generation and motivate alternative design choices. Overall, our work demonstrates that combining structured LLM generation with a robust execution and orchestration layer is critical for building practical, scalable agentic workflow systems.

Features implemented

Requirement 1:

- Core: NLP - Workflow Generation
- Bonus: Multi-round conversation to Update Workflow Graph
- Bonus: Editable Workflow Visualization

Requirement 2:

- Core: Node-level capabilities implementation
- Core: Workflow level concerns with dependency, parallelism and recovery
- Bonus: Distributed execution

- Bonus: Multi-tenancy support

Requirement 3:

- Core: End-to-end demo

Introduction

Recent advances in agentic artificial intelligence (Agentic AI) have fundamentally transformed how users interact with large language models (LLM). Top-tier models like GPT-5 and Qwen have demonstrated strong capabilities in natural language understanding, reasoning, structured output generation and tool calling (Yang et al., 2024). Consequently, there has been a shift from simple textual and conversational interactions with LLMs, towards intent-driven automations, where users express a desired intent in natural language, and the system executes and achieves that intent. This shift transforms LLMs from passive assistants to active systems capable of orchestrating complex, multi-step tasks.

To facilitate this growing demand for workflow automation, platforms like n8n have become increasingly popular, due to its numerous integrations and convenience in orchestrating complex, multi-step pipelines like Extract-Transform-Load (ETL). These platforms represent workflows as directed graphs, consisting of nodes that represent specific operations such as API calls, file I/O, and tool invocations. This abstraction enables applications to move beyond passive assistance to autonomous workflow execution, significantly expanding the practical utility of LLMs. In addition, platforms like n8n also provide a visual interface for designing workflows, increasing accessibility for users through a low-code and no-code (LCNC) paradigm. However, despite their popularity, existing systems have the following limitations.

Firstly, they primarily rely on manual workflow design. Users must explicitly define nodes, configure parameters, and connect dependencies through graphical interfaces. While powerful, this process is time-consuming, requiring users to understand the functionality and configuration of each node. This learning curve can be steep for non-technical users. Furthermore, as workflows become increasingly powerful and complex, it can be challenging and tedious to design, debug and maintain. Recent work has explored the use of LLMs to generate workflows from natural language intent (Li et al. 2024). The LLM takes the user request and produces a structured representation of the workflow, which will then be parsed

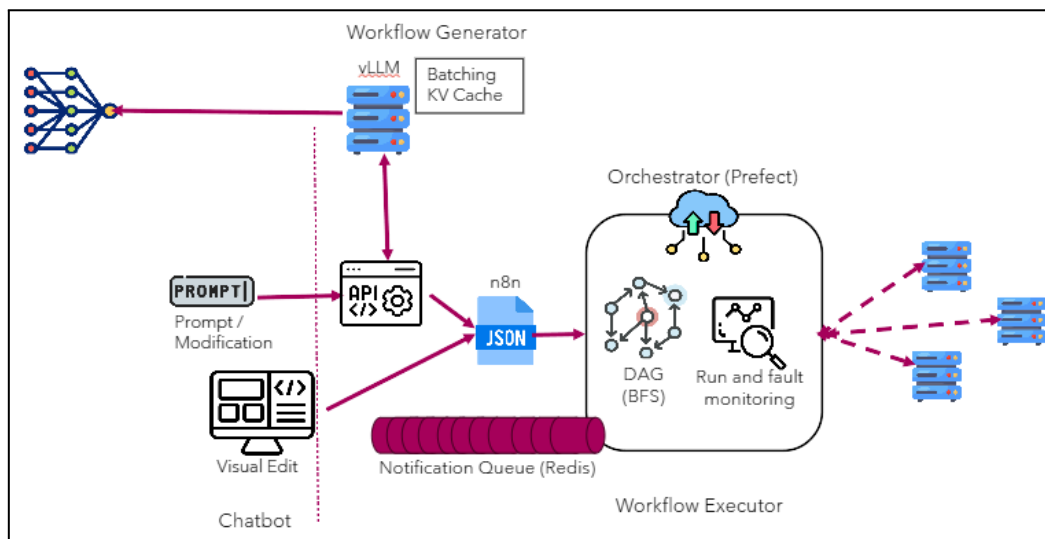
and executed. Such LLMs are often trained using supervised fine-tuning (SFT) on prompt-to-workflow pairs.

Secondly, existing workflow automation systems like n8n do not optimize execution. In these systems, workflows are typically treated as static, predefined graphs that are executed in a largely sequential and deterministic manner (n8n.io, 2024), without consideration for efficiency or runtime conditions. As a result, independent tasks that could be executed in parallel are often processed sequentially, leading to increased latency. These systems also lack adaptability during execution. They are unable to respond to failures adequately, stopping the entire workflow and leaving the responsibility of recovery or reruns to the user. Furthermore, there is no mechanism for global optimization, such as batching, across the workflow graphs. Consequently, workflow execution remains rigid and potentially inefficient, placing the burden of optimization, error handling, and performance tuning entirely on the user.

Given these limitations, we frame this work as a machine learning systems problem, rather than reimplementing existing workflow automation platforms like n8n. Specifically, we investigate how to bridge the gap between high-level user intent and efficient and reliable workflow execution via system improvements. Our submission explores the following features:

- 1) A LLM workflow generator that generates executable workflows through
 - a) Multi-turn inputs from the user, allowing iterative refinement of intent
 - b) Visual edits from the user, enabling human in the loop corrections
- 2) Workflow executor that supports
 - a) Dependency aware scheduling
 - b) Parallelism
 - c) Optimized multi-tenancy
 - d) Distributed execution
 - e) Fault tolerance

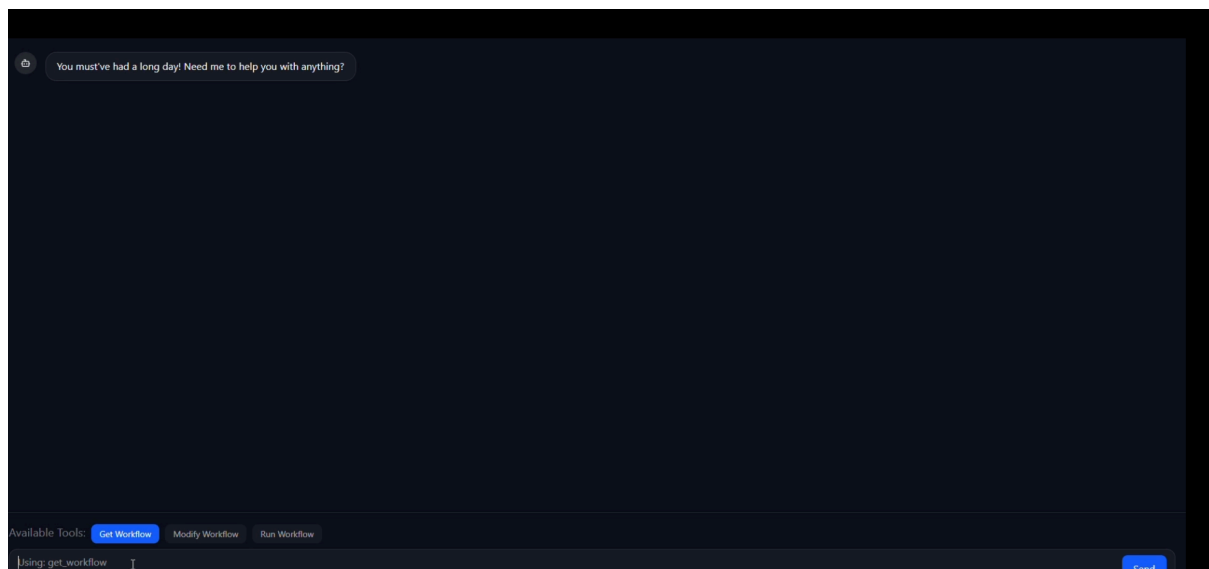
End to End System Design and Architecture



Architecture of our AI workflow application

Our workflow automation platform adopts a microservice-based architecture comprising 3 core services: A frontend chatbot service, a workflow generation service, and a workflow execution service. This modular design enables independent optimizations of each component, to improve scalability and efficiency.

Chatbot Service



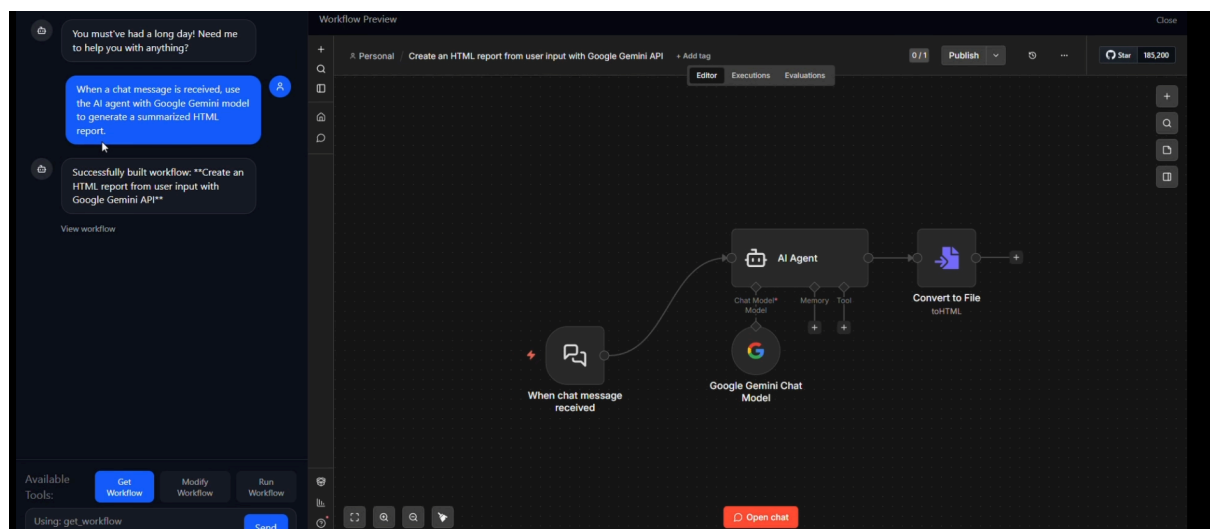
Screenshot of our chat interface

We use React as the chatbot frontend. Our choice of React as opposed to the originally provided chainlit frontend is motivated by the increased customization opportunities to fit our

chatbot and editable workflow viewing use case. Our chatbot serves as the primary interface between the user and the system, and supports 2 complementary interaction modes.

Firstly, users can express their automation intent in natural language. These requests are interpreted by the system either to generate a new n8n style workflow from scratch or to iteratively modify an existing workflow based on follow-up instructions. This enables a multi-turn interaction paradigm, where users progressively refine their workflows.

The multi-turn capability is implemented through a WorkflowManager data structure in the FastAPI backend, which stores a mapping from session_id to the latest workflow JSON for that user. When the user requests a modification, the existing workflow is retrieved, embedded into the editing prompt, allowing the system to update the workflow incrementally rather than regenerate it from scratch. This design preserves structural continuity, and improves alignment with the intent of the user.

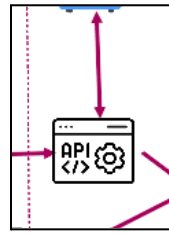


Visual Workflow Editor on the right panel

Secondly, the system provides a visual workflow editor that enables direct manipulation of nodes, parameters and connections. This human in the loop mechanism allows users to directly manipulate nodes, parameters and connections in the workflow graph. This visual editing feature complements the multi-turn conversations, providing greater control and modification transparency.

Combined, these 2 interaction modes enable a hybrid interaction approach that combines natural language conversations and direct workflow manipulation, providing more comprehensive, flexible, and precise workflow construction.

Backend Service



FastAPI Backend

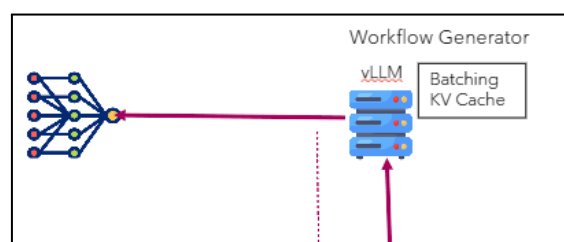
We have a backend server layer, implemented via FastAPI. FastAPI exposes 3 primary endpoints:

- 1) `/get_workflow`: For first-pass workflow generation
- 2) `/modify_workflow`: Session-aware workflow graph editing
- 3) `/run_workflow`: Workflow execution of finalized workflow model

This backend layer helps to bridge the interaction between the user on the frontend, the workflow generator, and execution engine.

Our backend FastAPI server can process requests and maintain state for multiple users concurrently, managing sessions and coordinating orchestration of other workflows being executed. Through the use of the same WorkflowManager data structure above, anytime a user's workflow is created or updated, this data structure will be updated. In a multi-request and multi-user setting, this would reduce end to end latency and thus improve responsiveness for each user.

Workflow Generation Service



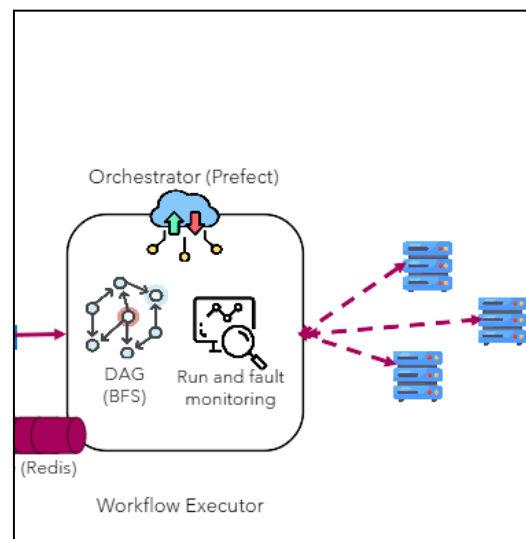
Separate vLLM server deployed in cloud (Vast AI)

Our workflow generation LLM component is deployed as a separate serving system from the FastAPI server, using vLLM. By decoupling the LLM inference from the FastAPI backend, we are able to support multi-tenancy effectively. Multiple incoming requests can be batched together efficiently, improving throughput. Additionally, KV caches can also be more

efficiently utilized within the same serving system, reducing latency for similar or repeated requests. At the same time, the FastAPI backend will not be bottlenecked by LLM inference.

Once the LLM generates the workflow, our generation service will perform a validation step to ensure the generated JSON is compliant with the n8n JSON schema. We perform a postprocessing step to add position coordinates to each node, to facilitate visualization in the frontend workflow editor. The positions are computed using a breadth-first traversal (BFS), so that each level in the graph (i.e. parallel branches) are visually aligned, improving interpretability.

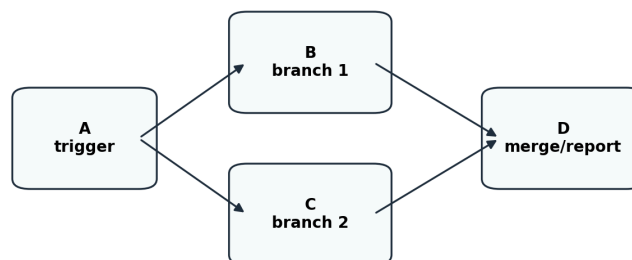
Workflow Execution Engine



Workflow Executor Engine

When the workflow is confirmed by the user, it is passed to the execution layer, which begins a series of steps as part of execution.

Dependency-aware scheduling and parallel execution



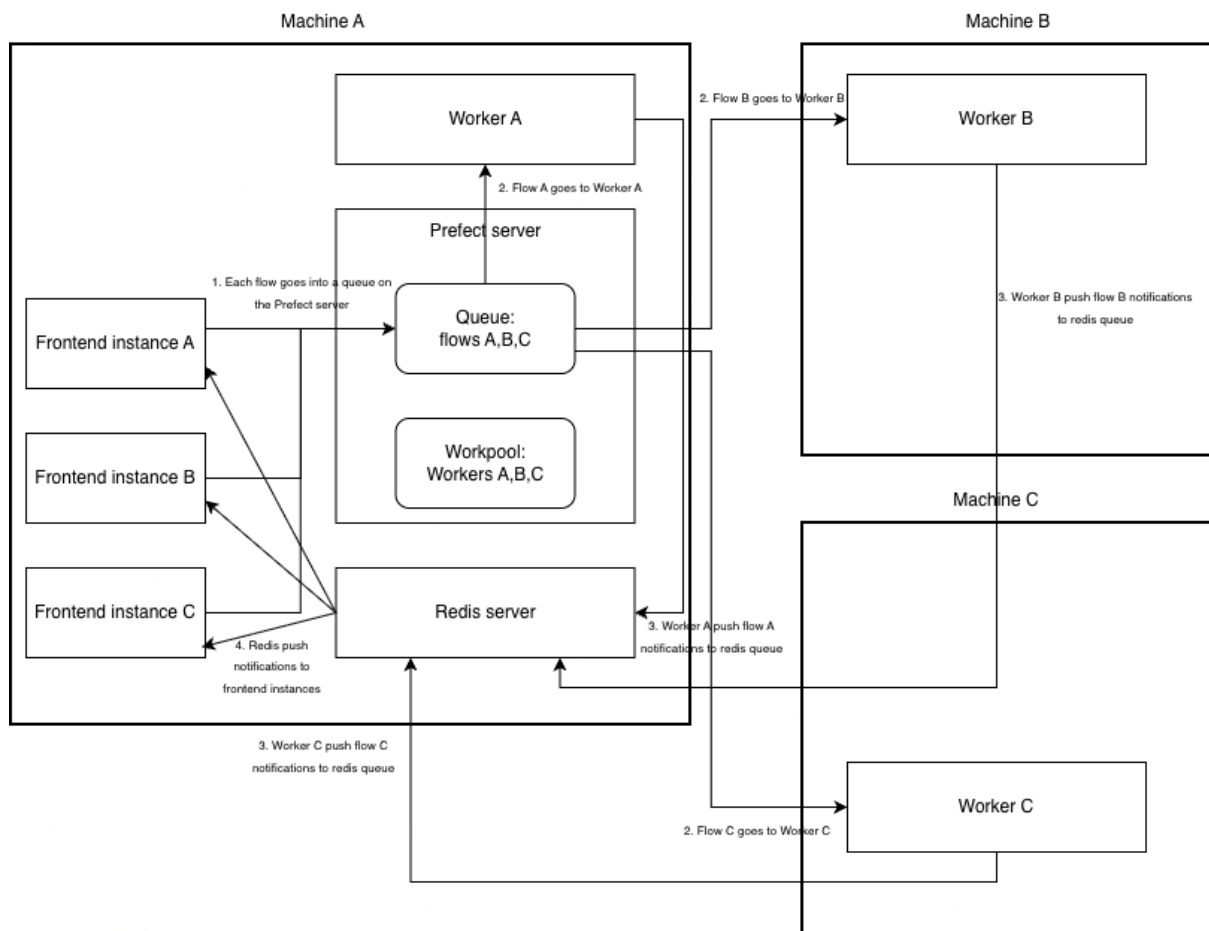
Dependency-aware scheduling used by the workflow runtime.

Firstly, the executor parses the workflow JSON into nodes and edges, constructing a dependency graph. This is done with a BFS traversal of the JSON. We then derive and build the execution order of the directed graph based on dependencies. To do this, we encode dependency information directly in the function arguments for each task. This allows Prefect to handle task scheduling based on the dependencies.

On the separate Prefect server, a Prefect deployment is used to receive and push the workflow into a queue. This queue is connected to a pool of multiple distributed workers. Any of the available workers in the work pool can pick up the workflow and run it asynchronously, to completion.

To provide real time feedback to the user on the frontend, we use Redis as a messaging layer. Specifically, we use a Redis queue to push notifications of completed nodes (Prefect tasks) from the Prefect server to the frontend to receive. We use a polling mechanism to receive notifications of Prefect futures completing to push the notifications into the redis queue as Prefect does not support waiting for each completed future, such as with ``asyncio.wait(FIRST_COMPLETED)``.

Prefect, Redis, and multi-tenancy distributed execution



Example of distributed Prefect workflow execution architecture.

A major systems contribution of the repository is the separation between request serving and workflow execution. FastAPI does not run workflows in-process. Instead, `/run_workflow` submits a Prefect flow run, subscribes to a Redis channel keyed by `session_id`, and streams NDJSON messages back to the frontend as workers publish node notifications.

The application is set up to be extensible with multiple workers in a distributed environment. By exposing the Prefect server and redis server, multiple workers can connect to these servers to pick up workflows that are pushed into a workpool and each worker can send the notifications of completed tasks via the respective Redis channel. We demonstrate this by exposing the servers with ngrok and using a `'start_worker.sh'` script to connect the worker to the servers. This can easily scale in other environments such as AWS cloud with more workers connected to a central Prefect and redis server.

This design has three benefits. First, it provides real-time observability: the user sees node-by-node progress rather than waiting for the entire workflow to finish. Second, it

provides fault isolation: a failure in one workflow run does not crash the API server or another user's session. Third, it enables distributed execution at the workflow-run granularity because multiple Prefect workers on one or more machines can pull from the same deployment queue.

Fault tolerance and error handling

The runtime supports fault tolerance and error handling at multiple levels, to ensure robust execution.

At the individual node level, each node function is wrapped as a Prefect task with retries and exponential backoff. This is well matched to transient failures such as HTTP timeouts, API rate limits, or short-lived infrastructure issues. If the particular node fails, the node task will be retried up to the set limit.

At the application infrastructure level, we deploy independent workers connecting to a central Prefect server to execute workflow runs. To anticipate worker crashes and system level failures, we run 2 “automations” (event-based triggers) on Prefect – a zombie flow automation to detect flow runs that have disconnected from their worker as well as a rerun crashed flow automation to pick up crashed flows and push it to the workpool for other workers to run it to completion. This enables fault tolerance in a distributed system. The zombie flow automation makes use of worker heartbeats (set using `PREFECT_FLOWS_HEARTBEAT_FREQUENCY=31`) to check for every workflow if the worker has sent a heartbeat within 2 heartbeat cycles. If the worker fails to submit a heartbeat within the timeframe, the automation will mark the workflow as crashed and the rerun crashed flow automation will send the crashed flow to the common workpool for other workers to execute it.

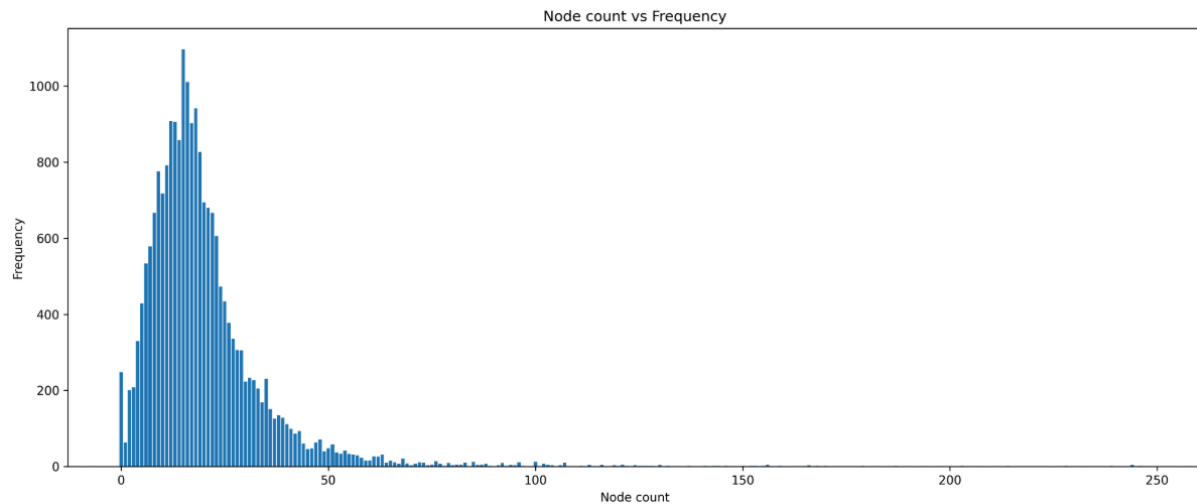
At the user-interface level, errors are surfaced as structured notifications rather than uncaught crashes, allowing the frontend to terminate gracefully with a final workflow-level status message.

Workflow Generator LLM Fine-tuning

Dataset construction

We first sourced for n8n style workflow JSONs from various sources, totalling to 21,236 workflows from various publicly available sources such as <https://n8n.io> and

<https://n8nflow.net>. To better understand the dataset characteristics, we performed exploratory data analysis (EDA) on workflow structure, including node counts, node types, and parameter distributions.



Distribution of node counts in each workflow

Our analysis revealed that many workflows contained non-functional components such as sticky note nodes, which serve only as user annotations and do not contribute to execution semantics. These nodes were removed to reduce noise in the training data. In addition, we excluded workflows exceeding 15 nodes to remain within the context length constraints of a 3B parameter model, while still preserving sufficient structural diversity. We also removed positional metadata associated with nodes, as these values are only relevant for visualization and do not affect execution. Instead, node positions are reintroduced programmatically during post-processing.

After preprocessing and filtering, the final dataset consisted of 2,988 workflows. To further align the dataset with our specific workflow domain: chatbot-style agentic workflows, we constructed a second, more specialized subset by retaining only workflows that include chat-based triggers or agent-related nodes. This resulted in a refined dataset of 509 workflows, focused on conversational and agentic use cases.

Variation 1: Create a chat bot that triggers when a message is received, utilizes an OpenAI Chat Model (gpt-4o-mini) with a simple memory buffer, and schedules Google Calendar events with the specified details using the googleCalendarOAuth2Api Credential.

Variation 2: Configure a workflow that receives chat messages, invokes an AI agent using the gpt-4o-mini model, and creates events in Google Calendar with the default Asia/Manila timezone, using the googleCalendarOAuth2Api Credential for authentication.

Example of generated prompts with variations in phrasings

In our next step of dataset construction, we used a Qwen3-8B model to synthesize prompts corresponding to each workflow JSON. This model was selected as a practical trade-off between generation quality and computational efficiency. For each workflow, we generated multiple prompt variants with diverse phrasings to reduce overfitting to specific lexical patterns. This augmentation improves the model’s robustness to varied user inputs.

To facilitate multi-turn conversations for editing the workflows, we further extended the dataset with multi-turn edit examples by asking for plausible modifications to an existing workflow, such as adding a filter node, changing a parameter, or inserting an additional action. Similar to the single-turn generation, we generated multiple possible workflow edit variations. The final corpus for supervised fine-tuning concatenates the single-turn generation examples with multi-turn modification examples. This design enables the model to support both initial workflow generation and iterative refinement.

Supervised Fine-tuning

We fine-tuned Qwen2.5-3B-Instruct as our base model for workflow generation. To make training feasible under limited computational resources, we adopted a parameter-efficient fine-tuning strategy using QLoRA, along with several system-level optimizations. Specifically, we employed FlashAttention-2 for efficient attention computation and Liger Kernel for optimized operator fusion. While these techniques do not directly improve task performance, they significantly reduce training cost and latency, enabling practical experimentation within course-scale constraints.

We also applied assistant-only loss masking, which helps restrict gradient updates only to the model’s output tokens. This encourages the model to focus on generating well-structured and valid n8n workflow JSONs, rather than memorizing prompt formats, leading to more stable and consistent outputs. The training configurations are summarized below.

Component	Configuration used in the project
Base model	Qwen2.5-3B-Instruct
Fine-tuning method	QLoRA
LoRA rank / alpha	32 / 64

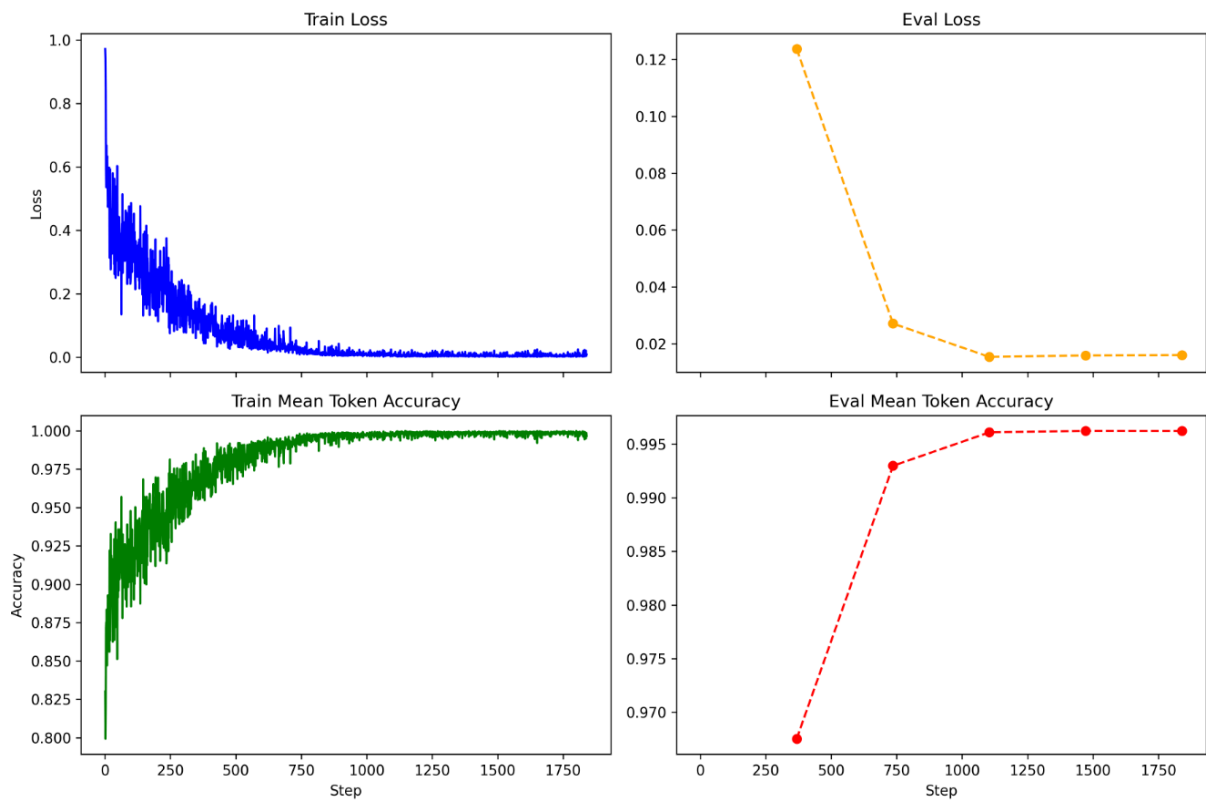
Epochs	8
Learning rate	2e-4 with cosine decay
Optimizer	AdamW 8-bit
Training optimizations	Assistant-only loss masking, FlashAttention-2, Liger Kernel

Fine-tuning configuration

Our fine-tuning process was conducted in 2 stages to progressively specialize the model. In the first stage, the model was trained for 5 epochs on the full, general purpose dataset of 2,988 workflows. This stage enables the model to learn broad but valid workflow structures and general node interactions. In the second stage, the model was further fine-tuned for 3 epochs on the specialised subset of 509 workflows focused on chat-triggered and agent-based patterns. This curriculum style-training approach biases the model towards our specific use case while retaining general workflow knowledge.

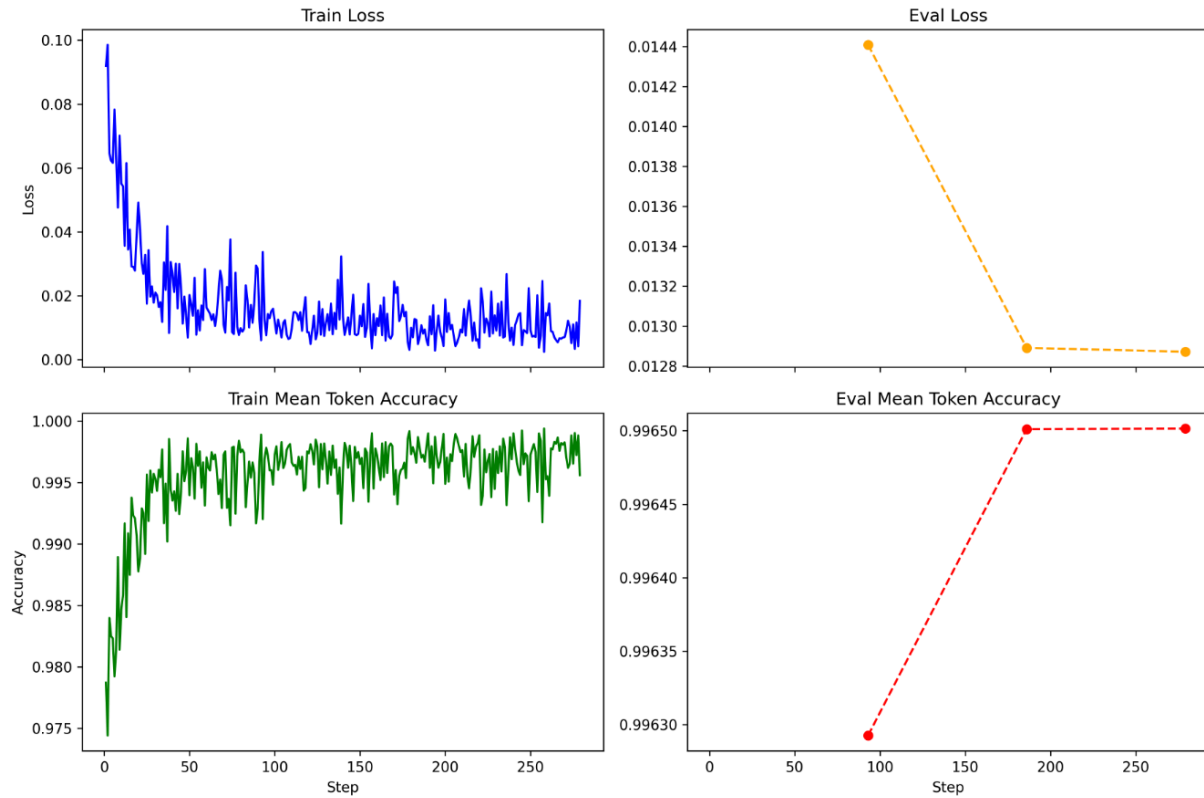
Evaluation

Fine-tuning



Training statistics phase 1

We evaluate the effectiveness of our fine-tuning pipeline across two training phases. In the first training phase, the training and validation loss and accuracy over the 5 epochs are shown above.



Training statistics phase 2

The training results for phase 2 over 3 epochs are shown above. Across both phases, training and validation metrics showed stable convergence, with no significant signs of overfitting.

json_validity	1.0
schema_compliance	1.0
graph_validity	0.943127962085308
node_precision	0.6977324939410248
node_recall	0.7277037470165433
type_accuracy	0.9588749717896637

connection_f1	0.539943873674948
---------------	-------------------

Evaluation results on a held-out test dataset

To assess generalization, we evaluated the final model on a held-out test set using structural and semantic metrics tailored to n8n workflows. Based on the above empirical results, we found that the model achieves perfect performance on JSON validity and schema compliance, indicating that it reliably generates syntactically correct and well-formed workflow structures. It also performs strongly on node type prediction, as reflected in the high type accuracy. However, performance degrades on connection-level metrics, with a relatively low connection F1 score. This suggests that while the model is capable of generating valid nodes, it struggles to correctly specify dependencies and relationships between them. As a result, some generated workflows are structurally valid but semantically incomplete or suboptimal.

To address these shortcomings, we explored an additional fine-tuning phase using Group Relative Policy Optimization (GRPO). The objective was to improve structural correctness by designing reward functions based on node validity, connection correctness, and parameter consistency.

In practice, this approach led to degraded performance. We attribute this to two key challenges. First, the variability of workflow structures makes reward design difficult. In n8n workflows, nodes can have optional or context-dependent parameters, and the importance of each parameter varies across node types. This makes it difficult to define a consistent reward function that appropriately penalizes incorrect outputs without suppressing valid variability. Second, the reward signals lacked semantic grounding. While the reward function captured structural properties such as valid nodes and connections, it did not reflect whether a workflow was actually executable or meaningful. As a result, the model could optimize for superficial correctness without improving functional quality.

Given these limitations, we chose not to adopt GRPO in the final system. Instead, we improved output quality through dataset refinement and system-level design choices, including multi-turn interaction and human-in-the-loop editing. As future work, a more robust reward framework could be explored, potentially incorporating execution-based feedback or auxiliary LLM evaluators to better capture semantic correctness.

Workflow Execution Engine

Our execution engine adopts a workflow-level distribution strategy rather than distributing individual nodes of a single workflow across multiple machines. Under this design, each workflow run is assigned to a worker, while parallelism is achieved within the workflow through concurrent execution of independent nodes.

Although node-level distribution was considered, we found it to be less suitable for our use case. Most workflows in our dataset are relatively small (typically 5–10 nodes) and operate over shared data. Distributing nodes across machines would introduce communication overhead and degrade performance due to reduced data locality. In contrast, workflow-level distribution offers a more practical balance between scalability and efficiency. It enables multi-user concurrency, process isolation, and straightforward deployment in distributed environments. For larger or more complex workflows, future work could explore hybrid strategies, where node-level distribution is selectively applied to computationally intensive or heterogeneous tasks. Such approaches could further improve scalability in more demanding settings.

In terms of parallelism, the Dual RSS Parallel ChatBot test case found in the repository is the strongest demonstration of runtime parallelism. Its graph contains two independent analysis branches that pull from different sources (OpenAI’s <https://openai.com/blog/rss.xml> and Google’s <https://blog.google/technology/ai/rss/>) and then synchronize downstream. Because the executor releases runnable nodes as soon as their dependencies are satisfied, both branches can proceed concurrently. This workflow is therefore particularly well suited to a demo that emphasizes systems behavior rather than only application semantics.

Implemented node library

The submitted runtime supports 21 node types, substantially expanding the expressivity of the starter kit. This matters both for execution and for generation: the fine-tuned model can only be useful to the extent that the runtime can execute the node patterns it predicts.

Category	Implemented node types
Triggers and control	chatTrigger, manualTrigger, scheduleTrigger, if, wait, noOp
Agentic reasoning	agent, lmChatDeepSeek, memoryBufferWindow, toolCode, outputParserStructured

Data access and integration	httpRequest, readBinaryFile, spreadsheetFile, googleSheets
Transformation and routing	set, code, function, functionItem, filter, merge
Artifact generation	convertToFile

Node categories implemented in the submitted execution engine.

Conclusion

This project transforms a basic workflow automation starter system into a more capable agentic workflow platform. The resulting system supports end-to-end automation: it accepts natural language input, generates or refines n8n-style workflows, visualizes execution graphs, and executes them with dependency-aware scheduling and parallelism while streaming real-time updates to the user. Beyond the baseline requirements, the system introduces several key capabilities, including multi-turn workflow generation, human-in-the-loop editing, multi-user concurrency, and distributed workflow execution. These features collectively improve both usability and system performance. From a machine learning systems perspective, the primary contribution lies in the integration of model generation with a robust execution and serving infrastructure. Rather than treating workflow generation as an isolated modeling task, this work demonstrates the importance of coupling structured generation with runtime systems that ensure correctness, efficiency, and observability.

Analysis and Evaluation of Other Groups during the Project Showcase

Agentic Workflows Group 7

This group stood out because their project had a much more clearly restricted target user group. Rather than trying to support a very general workflow generation system for many possible users and task types, their system was designed around enabling employees of a company with work-specific automation needs. This gave their project a clearer product boundary as they only needed to implement nodes and workflow patterns that were directly useful for that user group. In comparison, our own system attempted to support a broader

agentic workflow setting, which required us to implement a much larger range of possible nodes and workflow behaviours.

This was an important lesson for our group. In hindsight, a narrower user target group could have made our system easier to optimize. Since our workflow generator had to support many possible node types, the model needed to learn not only valid n8n-style JSON syntax, but also which nodes were appropriate, how they should be connected, and what parameters they required. This broader scope likely contributed to one of our main weaknesses: although our model achieved strong JSON validity and schema compliance, it was weaker at generating correct and complete workflow connections. A more focused user group could have reduced the search space and allowed us to curate a more specialized dataset, improving both generation accuracy and executability.

For an agentic workflow system, a smaller and more specific user domain may actually improve usability, because the system can anticipate common tasks, standardize node templates, and ask fewer unnecessary questions. If we were to extend our project, we could define a clearer initial user group, such as students automating research tasks, employees automating internal reporting, or administrators handling repeated document workflows. This would allow us to prioritize high-frequency workflows and avoid implementing many low-use nodes too early.

Agentic Workflows Group 16

Their most useful idea was their Clarification Agent. Their system did not immediately assume that the user's first prompt was sufficiently complete. Instead, the clarification stage asked the user to describe the task more precisely before the workflow was finalized. This is highly relevant to our project because natural-language workflow generation is very sensitive to missing details. For example, a user may say, "Summarize this file and send it to me," but the system still needs to know which file, what kind of summary, where to send the result, and whether the task should happen once or repeatedly.

This idea could directly improve our workflow generator. In our system, a clarification agent could be inserted before the workflow generation model as an intent-refinement layer. Instead of passing the raw user prompt directly to the model, the clarification agent could convert the prompt into a more structured intermediate specification. This specification could include the trigger, required inputs, tools, data sources, transformation steps, output format, and final

destination. Once these fields are filled, the workflow generator would receive a much cleaner prompt, making it easier to select the correct nodes and connect them properly.

This would also complement our existing multi-turn feature. Currently, our multi-turn workflow editing allows the user to correct or modify a generated workflow after the first version has been created. However, a clarification agent would shift some of that correction earlier in the pipeline. Instead of relying mainly on post-generation edits, the system could prevent many errors before generation begins. This is especially useful given that our evaluation showed weaker performance in connection generation. A clearer user prompt would make the intended dependency structure more explicit, helping the model link nodes more accurately.

LLM Serving Group 4

This was the group we found most impressive overall. What stood out was not just their final implementation, but their evidence-backed development process. They appeared to look at multiple research papers, compare possible approaches, and iterate toward the best design choice. This is something we could have done more systematically in our own project. Although our system had a strong implementation focus, especially in terms of workflow execution, distributed orchestration, and user interaction, our design decisions could have been better supported by a deeper literature review on agentic workflow generation.

For example, recent work such as AutoFlow studies automated workflow generation for LLM agents and explicitly addresses the difficulty of manually designing agent workflows (Li et al. 2024). It proposes both fine-tuning-based and in-context workflow generation methods, which is closely related to our own supervised fine-tuning approach for prompt-to-workflow generation. These papers could have helped us justify why dataset quality, workflow diversity, and API/node coverage matter so much for generation accuracy.

Another relevant paper is Benchmarking Agentic Workflow Generation, which introduces WorfBench and WorfEval to evaluate workflow generation using graph-aware methods such as subgraph matching (Qiao et al. 2025). This is especially relevant to our project because our model performed well on JSON validity but struggled more with workflow connections. A graph-based evaluation framework would have helped us measure not only whether the output was syntactically valid, but whether the generated workflow structure matched the intended task.

Our vote therefore goes to LLM Serving Group 4. Their project demonstrated the value of grounding system design in prior work, using research papers to guide implementation choices, and iterating based on evidence rather than intuition alone. For our own project, this suggests that future work should not only expand the node library or improve the executor, but also adopt a more research-driven methodology: compare workflow-generation architectures, use graph-aware evaluation metrics, benchmark against existing workflow-generation methods, and justify design choices through both experiments and literature. This would make the project stronger not just as a course implementation, but as a more convincing machine learning systems artifact.

References

- Li, Z., Xu, S., Mei, K., Hua, W., Rama, B., Raheja, O., Wang, H., Zhu, H., & Zhang, Y. (2024, July 1). *AutoFlow: Automated Workflow Generation for Large Language Model Agents*. arXiv.Org. <https://arxiv.org/abs/2407.12821v1>
- n8n.io. (2024, November 15). Is it possible to run a part of the workflow in parallel? N8n Community. <https://community.n8n.io/t/is-it-possible-to-run-a-part-of-the-workflow-in-parallel/60221/6>
- Qiao, S., Fang, R., Qiu, Z., Wang, X., Zhang, N., Jiang, Y., Xie, P., Huang, F., & Chen, H. (2025). *Benchmarking Agentic Workflow Generation* (arXiv:2410.07869). arXiv. <https://doi.org/10.48550/arXiv.2410.07869>
- Yang, A., Zhang, B., Hui, B., Gao, B., Yu, B., Li, C., Liu, D., Tu, J., Zhou, J., Lin, J., Lu, K., Xue, M., Lin, R., Liu, T., Ren, X., & Zhang, Z. (2024). Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. ArXiv.org. <https://arxiv.org/abs/2409.12122>

Appendix

Contribution

Weijie:

I was the overall team leader, responsible for setting the team's direction and preparation of submissions like video and demo day slides. I worked on the end to end fine-tuning, consisting of datasets collection, preprocessing, and supervised fine-tuning. I also explored RL-based fine-tuning approaches like GRPO to improve LLM workflow generation capabilities. I contributed to the design of our app's end-to-end system architecture, such as separating our LLM to run on a cloud vLLM, maximizing modularity, throughput of user requests, and efficiency of workflow execution where feasible. I also integrated the editable visual workflow panel into our custom-designed frontend. I also assisted with the implementation of the workflow execution engine, to include parallelism, distributed execution and fault tolerance.

Gary:

I mainly implemented and extended the execution engine of our project. I initially managed to implement the dependency resolution of workflow nodes as well as concurrent execution in a single machine using just asyncio operations. After that, together with Weijie, we explored different ways we could enhance our execution engine through parallel and distributed execution across numerous workers. We researched Temporal, Ray, Kubernetes and Prefect before settling on Prefect. I then iteratively developed on the distributed and parallel execution of workflows using Prefect, first defining the flow and nodes to run locally, then managing deployments and the distributed multi-worker setup, until finally implementing fault tolerance with Prefect automations.

Yuancheng

I extended the workflow engine by implementing and integrating several n8n-style nodes to improve the system's functional coverage and support a wider range of generated workflows. I also updated the node registry and ensured that each implementation followed the project's common input-output protocol so that the new nodes could execute consistently within the existing runtime. To validate these additions, I created and ran targeted smoke-test workflows through the engine, confirming correct behaviour for API calls, data transformation, programmable logic, natural triggered and database operations.

Yugan:

I implemented and extended several workflow nodes, helped align node parameter formats with our executor, and added supporting test cases and local test scripts to validate node behavior and end-to-end workflow execution. I also contributed to debugging and integration work, including environment setup, local execution, model-serving checks, and identifying workflow/runtime mismatches such as unsupported node types, incorrect parameter formats. In addition, I helped trial multiple candidate workflows for the live demo, especially around file-processing and news-summary examples.